UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

P2 - Uso de Smartphones y Riesgo de Adiccion

Hiter Frank Rubio Peralta
Alumno

Jossep Jorge Froilan Choroco Neyra
Alumno

Joel David De la Cruz Cardenas
Alumno

Rodrigo Octavio Lucatoni Flores Ruiz
Alumno

Victor Andres Diaz Garcia
Alumno

Curso:
Introduccion a Ciencia de Datos e Inteligencia Artificial

6 de Noviembre del 2026

CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

+51 936 603 314
pandascore@gmail.com

Tabla de contenidos

1. Datos y preparación **4**
1.1. Librerías . . . 4
1.2. Carga de datos . . . 4
1.3. Renombrado y dataset limpio . . . 4
1.4. Selección de features relevantes . . . 6
1.5. Medición de la fuerza predictora . . . 6
1.6. Decisiones de selección de variables . . . 8
1.7. Codificación de variables categóricas . . . 9
1.8. Normalización o estandarización . . . 9

2. División en entrenamiento, validación y prueba **9**

3. Modelos predictivos **10**
3.1. Selección de modelos . . . 10
3.2. Entrenamiento y ajuste de hiperparámetros . . . 10
3.3. Evaluación de modelos . . . 12

4. Explicabilidad del modelo **14**
4.1. Extracción de casos . . . 14
4.2. Análisis de probabilidades . . . 14
4.3. Justificación algorítmica paso a paso . . . 14

5. Autopsia del modelo **14**
5.1. DataFrame de errores . . . 14
5.2. Clasificación del error . . . 14
5.3. Análisis manual a nivel de registro . . . 14
5.4. Hipótesis y solución propuesta . . . 15

6. Comparación de modelos y conclusiones **15**
6.1. Comparación del rendimiento . . . 15
6.2. Ventajas, limitaciones y comportamiento de cada modelo . . . 15
6.3. Conclusiones generales . . . 15

7. Limitaciones **15**

Listado de Tablas

1. Descripción de variables creadas en la P1 . . . 5
2. Significado de cada variable del dataset. . . . 6
3. Primeras 5 filas del dataset. . . . 6
4. Decisiones sobre cada variable. . . . 8
5. Métricas de cada modelo en el conjunto de prueba, junto con la línea base (predecir siempre 'Adicto'). . . . 12

Listado de Figuras

1. Matrices de confusión de cada modelo en el conjunto de prueba. 14

1. Datos y preparación

1.1. Librerías

```
# ======================================================================
# CONFIGURACIÓN ÚNICA DEL REPORTE
# Librerías y estilos se ejecutan una sola vez
# ======================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, Latex
from scipy import stats
import math

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    accuracy_score,
    f1_score,
)

sns.set_theme(style="whitegrid")
```

Funciones de presentación heredadas de la P1: la función `tabla()` reproduce el estilo de tabla (`booktabs` con filas alternadas) directamente desde Python.

1.2. Carga de datos

Cargamos el dataset desde el directorio del proyecto.

```
df = pd.read_csv("datos/df_clean.csv")
```

El archivo entra completo: 7500 filas, 17 columnas y 0 nulos.

1.3. Renombrado y dataset limpio

Renombramos las columnas a inglés justo después de cargar, evitando caracteres complicados (`/`, `ñ`) en los nombres que usará el modelo:

```
df = df.rename(
    columns={
```

```
        "edad": "age",
        "genero": "sex",
        "horas_diarias_pantalla": "daily_screen_hrs",
        "horas_redes_sociales": "social_hrs",
        "horas_videojuegos": "gaming_hrs",
        "horas_trabajo/estudio": "work_study_hrs",
        "horas_sueño": "sleep_hrs",
        "notificaciones_por_dia": "daily_notifs",
        "aperturas_apps_por_dia": "daily_app_opens",
        "tiempo_pantalla_finde": "wknd_screen_hrs",
        "nivel_estres": "stress_level",
        "impacto_academico/laboral": "acad_work_impact",
        "etiqueta_de_adiccion": "addiction_label",
        "horas_ocio": "leisure_hrs",
        "ratio_finde_diario": "wknd_daily_ratio",
        "estado_adiccion": "addiction_status",
        "diferencia_finde": "wknd_diff",
    }
)
```

Se excluye la variable que representa la variable objetivo en formato string:

```
df = df.drop(columns=["addiction_status"])
```

La variable objetivo `addiction_label` queda solo su versión como 0/1, como la necesitan los modelos.

Las variables creadas en la P1 y su origen:

Tabla 1: *Descripción de variables creadas en la P1*

Variable	Origen	Significado
leisure_hrs	social_hrs + gaming_hrs	Horas de ocio: suma de horas en redes sociales y videojuegos.
wknd_daily_ratio	wknd_screen_hrs / daily_screen_hrs	Razón entre las horas de pantalla del fin de semana y las diarias.
wknd_diff	wknd_screen_hrs - daily_screen_hrs	Diferencia de horas de pantalla entre el fin de semana y un día n

Significado de cada variable del dataset:

Tabla 2: *Significado de cada variable del dataset.*

Variable	Significado
age	Edad del usuario en años.
sex	Género del usuario.
daily_screen_hrs	Horas diarias de uso de la pantalla.
social_hrs	Horas diarias en redes sociales.
gaming_hrs	Horas diarias de videojuegos.
work_study_hrs	Horas diarias de trabajo o estudio.
sleep_hrs	Horas de sueño diarias.
daily_notifs	Número de notificaciones recibidas por día.
daily_app_opens	Número de veces que se abren aplicaciones por día.
wknd_screen_hrs	Horas de uso de la pantalla el fin de semana.
stress_level	Nivel de estrés reportado.
acad_work_impact	Impacto del uso en el rendimiento académico o laboral.
addiction_label	Etiqueta binaria de adicción (variable objetivo).
leisure_hrs	Horas de ocio (social_hrs + gaming_hrs).
wknd_daily_ratio	Razón entre horas de pantalla diarias y de fin de semana.
wknd_diff	Diferencia de horas de pantalla: fin de semana menos día normal.

Tabla 3: *Primeras 5 filas del dataset.*

index	0	1	2	3	4
age	21	24	31	32	25
sex	Male	Other	Other	Other	Male
daily_screen_hrs	3.23	5.09	6.06	7.83	9.96
social_hrs	2.01	3.81	1.36	5.85	5.92
gaming_hrs	0.89	2.24	3.83	1.51	3.42
work_study_hrs	4.55	4.44	2.35	3.54	5.27
sleep_hrs	7.55	7.66	4.92	8.23	6.21
daily_notifs	248	127	44	178	136
daily_app_opens	154	71	106	107	177
wknd_screen_hrs	3.95	6.71	8.68	9.77	12.55
stress_level	Medium	Medium	High	High	Low
acad_work_impact	Yes	Yes	No	Yes	No
addiction_label	0	0	0	1	1
leisure_hrs	2.90	6.05	5.19	7.36	9.34
wknd_daily_ratio	1.22	1.32	1.43	1.25	1.26
wknd_diff	0.72	1.62	2.62	1.94	2.59

1.4. Selección de features relevantes

Tras el analisis exploratorio de los datos se definió que variables serían las protagonistas del modelo de predicción, pero se les presenta las metricas que fueron necesarias para está selección:

1.5. Medición de la fuerza predictora

Utilizamos la correlacion de punto biserial, que es un caso especial del coeficiente de correlacion de Pearson, está correlacion es perfecta para medir la fuerza predictora de una variable dado que

nuestra variable objetivo es de tipo binaria (Si/No), algo que el coeficiente de correlacion de Pearson no entiende.

```
df_sin_y = df.drop("addiction_label", axis=1)
resultados = []
for columna in df_sin_y:
    if pd.api.types.is_numeric_dtype(df[columna]):
        coeficiente, p_valor = stats.pointbiserialr(df["addiction_label"],
 ↪  df[columna])
        resultados.append(
            {
                "Variable": columna,
                "Coeficiente Correlación": coeficiente,
                "P-Valor": p_valor,
                "Significativa (p < 0.05)": "Sí" if p_valor < 0.05 else
                 ↪  "No",
            }
        )
df_correlaciones = pd.DataFrame(resultados)
df_correlaciones["Abs_Coeficiente"] = df_correlaciones["Coeficiente
 ↪  Correlación"].abs()
df_correlaciones = df_correlaciones.sort_values(
    by="Abs_Coeficiente", ascending=False
).drop(columns=["Abs_Coeficiente"])
tabla(df_correlaciones, indice=False)
```

Variable	Coeficiente Correlación	P-Valor	Significativa (p < 0.05)
daily_screen_hrs	0.58	0.00	Sí
wknd_screen_hrs	0.56	0.00	Sí
social_hrs	0.41	0.00	Sí
wknd_daily_ratio	-0.36	0.00	Sí
leisure_hrs	0.34	0.00	Sí
sleep_hrs	0.04	0.00	Sí
daily_app_opens	0.01	0.34	No
gaming_hrs	0.01	0.46	No
wknd_diff	0.01	0.63	No
age	0.00	0.71	No
work_study_hrs	0.00	0.92	No
daily_notifs	0.00	0.95	No

Como se puede observar en la tabla solo las primeras 5 variables son las que tienen una correlacion de punto biserial significante y por lo tanto una fuerza predictora en comparacion en las demás. Este es un gran indicador de qué debemos elegir.

Dado que el calculo anterior solo aplica para variables numericas, ahora mediremos la fuerza predictora de las variables categoricas a traves de chi cuadrado.

```
resultados_2 = []
for columna in df_sin_y:
```

```
    if (
        df[columna].dtype in ["object", "category", "bool"]
        or df[columna].nunique() < 10
    ):
        tabla_contingencia = pd.crosstab(df["addiction_label"], df[columna])
        chi2, p_val, dof, esperados =
↪   stats.chi2_contingency(tabla_contingencia)
        significativa = "Sí" if p_val < 0.05 else "No"
        resultados_2.append(
            {
                "Variable": columna,
                "Estadístico Chi2": round(chi2, 4),
                "Valor P": round(p_val, 4),
                "Grados de Libertad": dof,
                "Significativa (95%)": significativa,
            }
        )
df_resultados = pd.DataFrame(resultados_2)
df_resultados = df_resultados.sort_values(by="Valor P")
tabla(df_resultados, indice=False)
```

Variable	Estadístico Chi2	Valor P	Grados de Libertad	Significativa (95%)
sex	4.05	0.13	2	No
stress_level	0.43	0.80	2	No
acad_work_impact	0.00	1.00	1	No

Aqui se puede observar que ninguna variable categorica llega a tener un impacto para la predicción.

1.6. Decisiones de selección de variables

Tabla de decisiones.

Tabla 4: *Decisiones sobre cada variable.*

Variable	Decisión	Por qué
age	Descartar	No significativa, sin efecto
sex	Descartar	No significativa
daily_screen_hrs	Conservar	La más fuerte, efecto moderado-alto
social_hrs	Conservar	Efecto moderado
gaming_hrs	Descartar	No significante, sin efecto
work_study_hrs	Descartar	No significante, sin efecto
sleep_hrs	Descartar	Es significativa, pero irrelevante en la practica
daily_notifs	Descartar	No significante, sin efecto
daily_app_opens	Descartar	No significativa, sin efecto
wknd_screen_hrs	Descartar	Genera redundancia con daily_screen_hrs
stress_level	Descartar	No significativa, sin efecto
acad_work_impact	Descartar	No significativa, sin efecto
leisure_hrs	Descartar	Genera redundancia con social_hrs
wknd_daily_ratio	Conservar	El signo negativo no importa, solo la magnitud.Es informativa: quien usa mucho el celular
wknd_diff	Descartar	No significativa, sin efecto

1.7. Codificación de variables categóricas

Ya elegidas nuestras variables, vemos que son todas numéricas y es por ello que no necesitamos codificarlas.

1.8. Normalización o estandarización

¿A que modelos es recomendable aplicar una estandarización?

Modelos	¿Es necesario estandizarlas?
Regresión logística	Es necesaria
Random forest de clasificación	No es necesaria
XGBoost con árboles, para clasificación	No es necesaria
Árbol de decisión de clasificación	No es necesaria

2. División en entrenamiento, validación y prueba

Se decidió utilizar una división de 70 %, 15 % y 15 %; un 70 % dirigido hacia el entrenamiento, un 15 % para las validaciones y un 15 % para el *testing*. Esta distribución permite que el modelo tenga la suficiente cantidad de registros para poder aprender y a su vez, manteniendo un conjunto de registros para las pruebas lo suficiemente amplio para poder evaluar el modelo.

```
# 1. separamos el 15% para test
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
)
# 2. separamos el 85% restante tomamos el +17.65% para validación (0.85 *
 ↪  0.17647 = 0.15 del total)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.17647, random_state=42,
 ↪  stratify=y_train_val
)
```

Ahora recién podemos aplicar la estandarización porque definimos X_train y X_test

```
escalador = StandardScaler()

X_train_l = pd.DataFrame(
    escalador.fit_transform(X_train), columns=X_train.columns,
 ↪  index=X_train.index
)
X_val_l = pd.DataFrame(
    escalador.transform(X_val), columns=X_val.columns, index=X_val.index
)

X_test_l = pd.DataFrame(
    escalador.transform(X_test), columns=X_test.columns, index=X_test.index
)

display(X_train_l.head())
```

	daily_screen_hrs	social_hrs	wknd_daily_ratio
1412	0.983485	-0.675399	-1.295336
6320	-0.058107	-1.203234	0.224126
3673	-1.284188	-0.304658	-0.019486
5181	0.618351	1.172024	-0.782125
520	0.656786	-0.681683	-1.183531

3. Modelos predictivos

3.1. Selección de modelos

Se eligieron los modelos de *Logistic Regression* y *Decision Tree*, porque ambos son adecuados para clasificación binaria y permiten explicar su predicciones de manera accesible. El modelo *Logistic Regression* permite encontrar una relación aproximadamente lineal entre las variables y la probabilidad de adicción. El modelo *Decision Tree* permite observar reglas de desición mediante condiciones sobre las variables. Se eligieron estos modelos para comparar un método lineal con uno basado en reglas.

3.1.1. Modelo 1:

Logistic Regression, es un modelo lineal fácil de interpretar. Sin embargo no es el modelo más potente de todos, pero nos va ayudar a poder dar una mayor explicabilidad en nuestro modelado

3.1.2. Modelo 2:

Decision Tree, es un modelo versátil, ya que puede hacer tareas de regresión o clasificación. Este modelo presenta la propiedad de ser fácil de comprender y generalmente tener un buen desempeño en el modelado.

3.2. Entrenamiento y ajuste de hiperparámetros

se usará grid search para ajustar los hiperparámetros, ya que el datset trabajado es pequeño y contamos ocn suficiente potencia computacional.

3.2.1. Modelo 1

```
# Estandarizamos entrenamiento + validación (mismo orden que y_train_val)
X_train_val_l = pd.DataFrame(
    escalador.transform(X_train_val), columns=X_train_val.columns,
 ↪  index=X_train_val.index
)
```

```
#1. Creamos una instancia del modelo base (Logistic Regression)

base_logi = LogisticRegression(max_iter = 1000, random_state = 42)

#2. Creamos la malla de hiperparámetros (con los parámetros más comúnes,
 ↪  pero en distintas perspectivas)

param_grid_logi ={
"C": [0.01, 0.1, 1.0, 10.0, 100.0],
```

```
 "solver" : ["liblinear"] #se eligió este, porque soporta "l1" y"l2
 }

#3. Configuramos grid Search (Para ello creo una instancia de GridSearch)

grid_logi = GridSearchCV(
    estimator = base_logi,
    param_grid = param_grid_logi,
    cv = 5,
    scoring = "f1_macro",
    n_jobs = -1,
)

#4. Entrenamos el Grid Search (contiene nuestro modelo) con los datos de
 ↪  entrenamiento y validación, ya que internamente GridSearch lo reparte
grid_logi.fit(X_train_val_l, y_train_val)
```

```
print("Mejores Hiperparámetros:", grid_logi.best_params_)
print("Mejor F1-Score en CV:", round(grid_logi.best_score_, 4))
logi_reg_best = grid_logi.best_estimator_
```

```
Mejores Hiperparámetros: {'C': 0.1, 'solver': 'liblinear'}
Mejor F1-Score en CV: 0.8684
```

Finalmente con Grid Search, obtuvimos los mejores hiperparámetros para el modelo.

3.2.2. Modelo 2

```
#1. Creamos la instancia con el modelo Decision Tree
base_tree = DecisionTreeClassifier(random_state=42)

#2. Creamos la malla de hiperparámetros
param_grid_tree = {
    'max_depth': [None,2, 3, 5, 10, 15],
    'min_samples_split': [2,3,4, 5, 10],
    'criterion': ['gini', 'entropy']
}

# 3. configuramos GridSearch
grid_tree = GridSearchCV(estimator=base_tree, param_grid=param_grid_tree,
 ↪  cv=5, scoring='f1_macro', n_jobs=-1)

#4 Entrenamos el GridSearch (contiene nuestro modelo), con los datos sin
 ↪  estandarización.
grid_tree.fit(X_train_val, y_train_val)
```

```
print("Mejores Hiperparámetros:", grid_tree.best_params_)
print("Mejor F1-Score en CV:", round(grid_tree.best_score_, 4))
```

```
tree_best = grid_tree.best_estimator_
```

3.3. Evaluación de modelos

Luego del entrenamiento nos hemos quedado con los siguientes dos modelos: `logi_reg_best`: *Logistic Regression* y `tree_best`: *Decision Tree*. Guardamos las predicciones en las variables: `y_pred_logi` y `y_pred_tree`.

3.3.1. Accuracy

Modelo	Accuracy
Regresión Logística	0.8951
Árbol de Decisión	0.9396
Línea base	0.7076

3.3.2. F1-Score

```
from sklearn.metrics import precision_score, recall_score

predicciones = {
    "Regresión Logística": y_pred_logi,
    "Árbol de Decisión": y_pred_tree,
    "Línea base": baseline,
}
resumen_prueba = pd.DataFrame([
    {
        "Modelo": nombre,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1-Score": f1_score(y_test, pred),
    }
    for nombre, pred in predicciones.items()
])

tabla(resumen_prueba, decimales=4, indice=False)
```

Tabla 5: *Métricas de cada modelo en el conjunto de prueba, junto con la línea base (predecir siempre 'Adicto').*

Modelo	Accuracy	Precision	Recall	F1-Score
Regresión Logística	0.8951	0.9124	0.9422	0.9271
Árbol de Decisión	0.9396	0.9986	0.9158	0.9554
Línea base	0.7076	0.7076	1.0000	0.8287

Comparación directa de F1-Score durante la validación cruzada (solo referencia, no es la métrica de prueba):

```
resumen_cv = pd.DataFrame({
    'Modelo': ['Regresión Logística', 'Árbol de Decisión'],
    'Mejor F1-Score (CV)': [grid_logi.best_score_, grid_tree.best_score_]
})

tabla(resumen_cv)
```

index	Modelo	Mejor F1-Score (CV)
0	Regresión Logística	0.87
1	Árbol de Decisión	0.93

3.3.3. Matriz de confusión

```
labels = ["No adicto", "Adicto"]
models = [("Regresión Logística", y_pred_logi), ("Árbol de Decisión",
 ↪  y_pred_tree)]
row_no_addict = np.array([[True, True], [False, False]])  # fila 0 = No
 ↪  adicto

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, (name, pred) in zip(axes, models):
    cm = confusion_matrix(y_test, pred)
    # Verde para no adictos, rojo para adictos: más casos = color más
     ↪  intenso
    sns.heatmap(cm, mask=~row_no_addict, cmap="Greens", vmin=0,
 ↪  vmax=cm.max(),
                annot=True, fmt="d", cbar=False, ax=ax)
    sns.heatmap(cm, mask=row_no_addict, cmap="Reds", vmin=0, vmax=cm.max(),
                annot=True, fmt="d", cbar=False, ax=ax)
    ax.set_xticklabels(labels)
    ax.set_yticklabels(labels)
    ax.set_title(name)
    ax.set_xlabel("Predicción")
    ax.set_ylabel("Real")
plt.tight_layout()
plt.show()
```

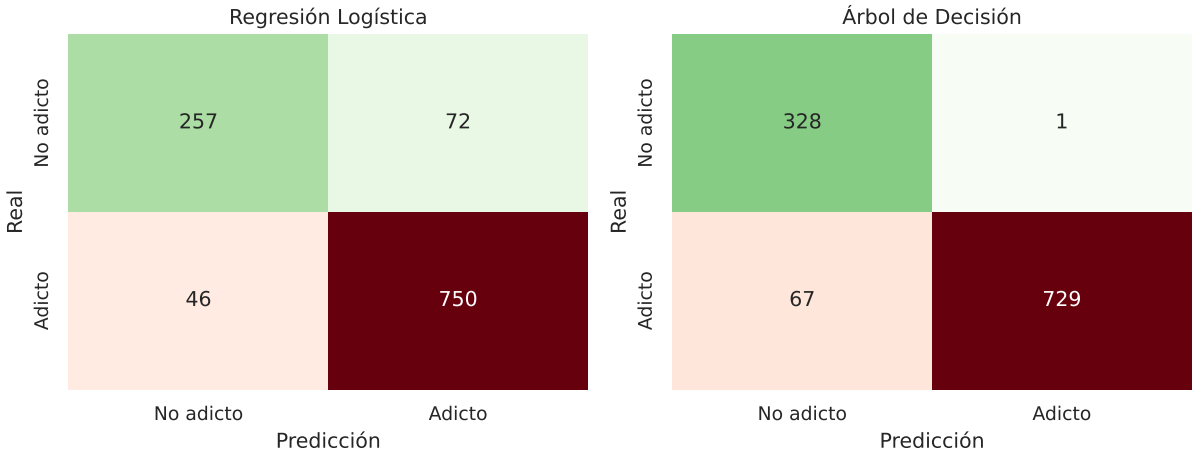


Figura 1: *Matrices de confusión de cada modelo en el conjunto de prueba.*

4. Explicabilidad del modelo

Toda esta sección se hace sobre el **mejor modelo** según la evaluación anterior.

4.1. Extracción de casos

```
# TODO: seleccionar 3 registros de X_test bien clasificados e imprimir sus
↪  variables
```

4.2. Análisis de probabilidades

```
# TODO: predict_proba de los 3 registros
```

4.3. Justificación algorítmica paso a paso

```
# TODO: gráfico del árbol / coeficientes / vecinos más cercanos, según el
↪  modelo
```

5. Autopsia del modelo

5.1. DataFrame de errores

```
# TODO: DataFrame con exclusivamente las filas donde y_pred != y_test
```

5.2. Clasificación del error

```
# TODO: columna con el tipo de error (FP / FN) y conteo de cada uno
```

5.3. Análisis manual a nivel de registro

```
# TODO: mostrar las 5 filas seleccionadas
```

5.3.1. Caso 1

5.3.2. Caso 2

5.3.3. Caso 3
5.3.4. Caso 4
5.3.5. Caso 5
5.4. Hipótesis y solución propuesta
6. Comparación de modelos y conclusiones
6.1. Comparación del rendimiento

```
# TODO: tabla/figura comparativa final
```

6.2. Ventajas, limitaciones y comportamiento de cada modelo
6.3. Conclusiones generales
6.3.1. Impacto de los datos en el rendimiento
6.3.2. Mejoras de datos evidenciadas en la autopsia
6.3.3. Consideraciones éticas y de impacto real
7. Limitaciones

Referencias a usar: Kwon et al. (2013); Csibi et al. (2018).

Referencias

Csibi, S, MD Griffiths, B Cook, Z Demetrovics y A Szabo (2018). The Psychometric Properties of the Smartphone Application-Based Addiction Scale (SABAS). *International Journal of Mental Health and Addiction* **16**(2), 393-403.

Kwon, M, JY Lee, WY Won, JW Park, JA Min, C Hahn, X Gu, JH Choi y DJ Kim (2013). Development and Validation of a Smartphone Addiction Scale (SAS). *PLOS ONE* **8**(2), e56936.